

Strutture dati dinamiche

I limiti degli array

Abbiamo visto che per memorizzare più valori all'interno di un unico contenitore ed evitare di dichiarare più variabili è possibile utilizzare gli array. Tuttavia, questa struttura dati presenta una serie di limitazioni.

Dimensione fissa

Tanto per cominciare, hanno una **dimensione fissa**: quando si dichiara un array, dobbiamo per forza specificare una dimensione, che non può cambiare durante l'esecuzione del programma.

Questo rende gli array idonei solamente nel caso in cui sappiamo già a priori il numero di elementi che dobbiamo memorizzare ed il loro numero non cambia nel corso dell'esecuzione; generalmente, però, il numero di dati che ci interessa memorizzare non è noto, o comunque può aumentare (o diminuire) dinamicamente.

Esempio:

```
int[] numeri = new int[3];
```

Se volessimo aggiungere un quarto elemento dovremmo necessariamente creare un nuovo array di dimensione maggiore, copiarci dentro tutti gli elementi del nostro array e aggiungere il nuovo elemento. Chiaramente una gestione di questo tipo è scomoda.

Eliminazione complicata

Anche l'eliminazione di un elemento in mezzo all'array non è semplice, in quanto occorre poi spostare manualmente tutti gli altri elementi in esso contenuti per non lasciare un "buco".

In sostanza, gli array sono validi solamente quando sappiamo già in partenza quanti elementi occorre salvare e durante l'esecuzione del programma il loro numero rimane invariato.

Introduzione alle collezioni

Prima di vedere alcune strutture dati dinamiche, è opportuno dire che queste fanno parte di un framework di Java: le collezioni.

Questo framework contiene una serie di interfacce (ad esempio: List, Set e Map) e delle classi che le implementano (ad esempio: ArrayList, HashSet e HashMap), tutte facenti parte del package *java.util*.

Queste strutture dati sono dinamiche e ci consentono di memorizzare, cercare, ordinare ed organizzare i dati in maniere più semplice ed efficiente, grazie ai metodi standardizzati che mettono a disposizione.

La tabella seguente elenca le strutture dati più comuni.

Interfaccia	Classe
List	ArrayList
	LinkedList
Set	HashSet
	TreeSet
	LinkedHashSet
Map	HashMap
	TreeMap
	LinkedTreeMap

Idee generali:

- usare le classi che implementano l'interfaccia List quando ci interessa mantenere un ordine tra gli elementi, vogliamo fornire la possibilità di avere dei duplicati e vogliamo poter accedere agli elementi tramite il loro indice;
- usare le classi che implementano l'interfaccia Set quando ci interessa memorizzare solamente elementi univoci;
- usare le classi che implementano l'interfaccia Map quando vogliamo memorizzare coppie chiave-valore.

Le ArrayList

Anche ArrayList, esattamente come gli array, è una struttura dati che ci consente di utilizzare un unico contenitore per memorizzare più elementi; la principale differenza, che ne rappresenta anche un vantaggio, rispetto agli array è il fatto che la sua dimensione è dinamica. Questo significa che non abbiamo la necessità di specificare una dimensione della nostra lista.

Questo rappresenta un enorme vantaggio, che risolve i problemi emersi con l'utilizzo degli array.

Per poter utilizzare questa struttura dati è necessario importare la classe ArrayList dalla libreria java.util:

```
import java.util.ArrayList;
```

Sintassi:

```
ArrayList<Tipo> nome = new ArrayList<>();
```

Esempio:

```
ArrayList<Integer> numeri = new ArrayList<>();
```

Vedendo l'esempio può sorgere spontanea una domanda: come mai utilizziamo *Integer* e non *int*?

Per farla breve, *ArrayList* è una classe che contiene esclusivamente oggetti, ovvero tipi di dato non primitivi. Per questo motivo non possiamo utilizzare *int* (che è un tipo primitivo), ma dobbiamo utilizzare *Integer*. Nel nostro esempio non stiamo memorizzando direttamente dei numeri, ma degli oggetti che a loro volta contengono dei numeri.

Quindi con *ArrayList*, al posto di un tipo primitivo, dobbiamo usare il Wrapper corrispondente.

Ora che sappiamo cosa è *ArrayList* vediamo come utilizzarne una. Come per qualsiasi oggetto, anche uno di tipo *ArrayList* mette a disposizione dei metodi che possiamo utilizzare.

Inserire un elemento

Il primo passo è capire come poter inserire un elemento all'interno della nostra lista; per farlo, abbiamo a disposizione il metodo `add()`, presente in due varianti.

Sintassi 1:

```
nome.add(elemento);
```

In questo modo andremo ad aggiungere il nostro elemento (oggetto) in fondo alla lista.

Sintassi 2:

```
nome.add(indice, elemento);
```

Così facendo andiamo a specificare in qualche posizione vogliamo inserire il nostro elemento. Cosa molto importante: l'inserimento fatto in questo modo NON elimina l'elemento già presente in quella posizione, semplicemente verranno "spostati di 1 posizione" tutti gli elementi a partire dall'indice specificato.

Esempio:

```
ArrayList<Integer> numeri = new ArrayList<>();

numeri.add(5);
numeri.add(7);
numeri.add(2);

System.out.println(numeri); //output: [5, 7, 2]
```

```
numeri.add(1, 12);
```

```
System.out.println(numeri); //output: [5, 12, 7, 2]
```

Rimuovere un elemento

Abbiamo la possibilità di eliminare un elemento da una ArrayList, tramite il metodo `remove()`.

Sintassi:

```
nome.remove(indice);
```

Esempio:

```
ArrayList<Integer> numeri = new ArrayList<>();
```

```
numeri.add(5);
```

```
numeri.add(7);
```

```
numeri.add(2);
```

```
System.out.println(numeri); //output: [5, 7, 2]
```

```
numeri.add(1, 12);
```

```
System.out.println(numeri); //output: [5, 12, 7, 2]
```

```
numeri.remove(2);
```

```
System.out.println(numeri); //output: [5, 12, 2]
```

Possiamo anche cancellare tutti gli elementi della lista (svuotarla) tramite il metodo `clear()`.

Sintassi:

```
nome.clear();
```

Ottenere la dimensione della lista

Come abbiamo detto, una ArrayList non ha una dimensione fissa, ma cambia dinamicamente aggiungendo/rimuovendo elementi. Per sapere quanti elementi sta attualmente contenendo (la sua dimensione), possiamo usare il metodo `size()`.

Sintassi:

```
nome.size();
```

Ottenere un elemento della lista

Chiaramente dobbiamo avere la possibilità di accedere agli elementi contenuti all'interno di una ArrayList; per farlo, possiamo sfruttare il metodo `get()` specificando la posizione (l'indice) dalla quale recuperare l'elemento.

Sintassi:

```
nome.get(indice);
```

Esempio:

```
ArrayList<Integer> numeri = new ArrayList<>();

numeri.add(5);
numeri.add(7);
numeri.add(2);

System.out.println(numeri.get(0)); //output: 5
```

Modificare un elemento della lista

Se ci accorgiamo che un elemento della lista non è quello desiderato, possiamo modificarlo con il metodo `set()`, specificando l'indice ed il nuovo elemento.

Sintassi:

```
nome.set(indice, nuovoElemento);
```

Esempio:

```
ArrayList<Integer> numeri = new ArrayList<>();

numeri.add(5);
numeri.add(7);
numeri.add(2);

numeri.set(0, 6);

System.out.println(numeri.get(0)); //output: 6
```

Elenchiamo nella tabella seguente i metodi più utilizzati con le ArrayList.

Metodo	Descrizione	Tipo di ritorno
add()	Aggiunge un elemento alla lista	boolean
clear()	Rimuove tutti gli elementi della lista	void
contains()	Controlla se un elemento è presente nella lista	boolean (true se l'elemento è presente, false altrimenti)
get()	Restituisce l'elemento alla posizione specificata	Dipende dal tipo di elementi contenuti
indexOf()	Restituisce l'indice della prima occorrenza dell'elemento specificato	int
isEmpty()	Indica se la lista è vuota oppure no	boolean (true se la lista è vuota, false altrimenti)
lastIndexOf()	Restituisce l'indice dell'ultima occorrenza dell'elemento specificato	int
remove()	Rimuove un elemento dalla lista	boolean
set()	Modifica un elemento della lista	Dipende dal tipo di elementi contenuti
size()	Restituisce il numero di elementi della lista	int

Ciclare una ArrayList

Possiamo scorrere gli elementi di una ArrayList in due modi:

1. attraverso un ciclo for, usando il metodo size() per specificare il numero di iterazioni;
2. attraverso un ciclo for-each.

Esempio:

```
ArrayList<Integer> numeri = new ArrayList<>();

numeri.add(5);
numeri.add(7);
numeri.add(2);

for(int i = 0; i < numeri.size(); i++)
    System.out.println(numeri.get(i));

for(Integer n : numeri)
    System.out.println(n);
```